

Async and asyncio

Проблема sync

sync I/O:

- один поток обрабатывает запрос
- при ожидании I/O (сеть, БД) поток блокируется
- CPU простаивает
- другие запросы не обрабатываются

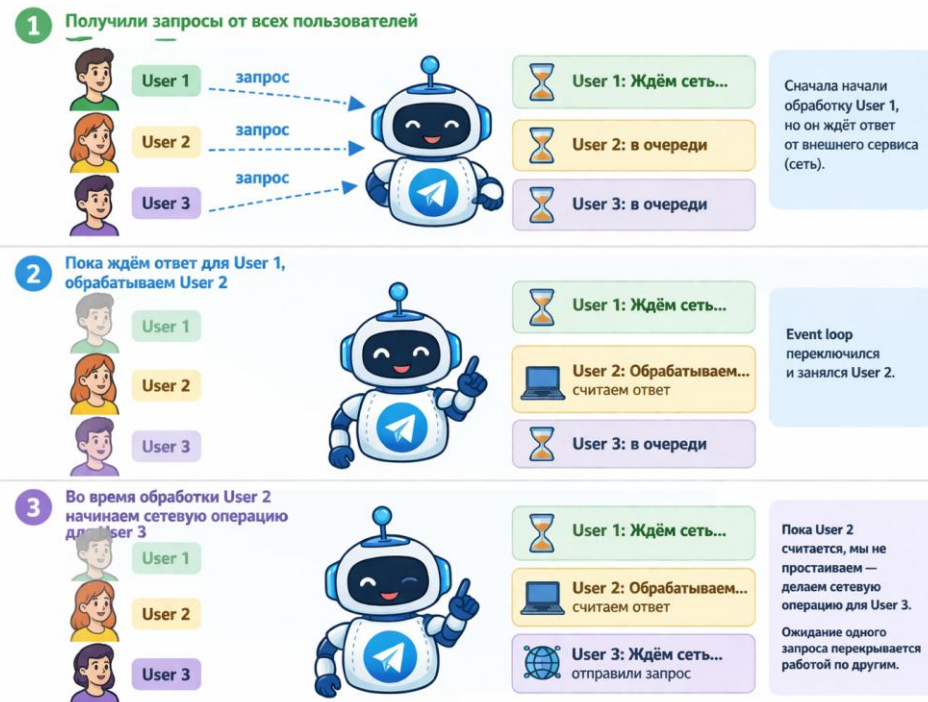
Идея №1 – использовать пул потоков

Проблема – если запросов/потоков становится слишком много, то возникают накладные расходы при переключении контекста со стороны ОС

Идея №2 – сделать ожидание неблокирующим:

- не “спать” в I/O
- а продолжать выполнять другие задачи
- возвращаться к запросу, когда данные готовы

Асинхронность: перекрываем ожидание полезной работой



Ресар. Сокет

Сокет — это канал общения с другим процессом через сеть, объект ОС

Файловый дескриптор (fd) — это число, через которое процесс обращается к ресурсам (файл, сокет), управляемым ядром ОС

```
import socket
```

```
# 1. создаем TCP-сокет
```

```
server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
# 2. настраиваем опции сокета
```

```
server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
# 3. привязываем сокет к адресу и порту
```

```
server_sock.bind(("0.0.0.0", 8080))
```

```
# 4. переводим сокет в режим ожидания входящих подключений
```

```
server_sock.listen()
```

```
# 5. принимаем клиентское соединение
```

```
client_sock, addr = server_sock.accept()
```

```
# 6. читаем данные от клиента
```

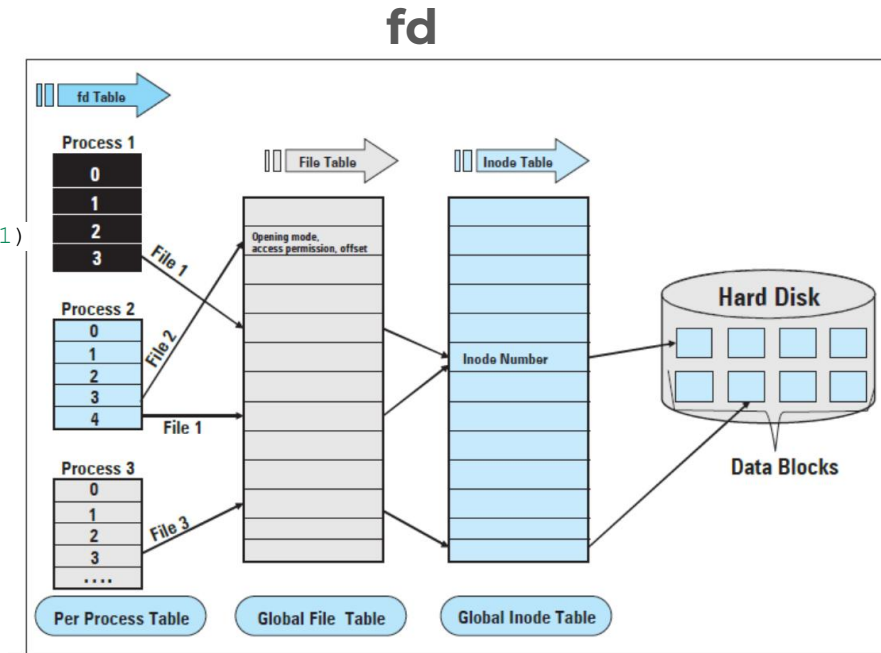
```
data = client_sock.recv(1024)
```

```
# 7. отправляем данные клиенту
```

```
client_sock.send(b"HTTP/1.1 200 OK\r\n\r\nHello")
```

```
# 8. закрываем сокеты
```

```
client_sock.close(), server_sock.close()
```



Продолжение идеи async

- В concurrency режиме в рамках ОДНОГО потока обрабатываем несколько запросов в сокет – неблокирующий I/O. Сюда можно отнести сетевые запросы: HTTP, Websocket, БД, сам сокет
- Но асинхронность – это не всегда про сетевое I/O, и в целом может быть полезна в задачах с **ожиданием**
- Например, асинхронным может быть также взаимодействие между процессами, только не через сокет, а через пайп
- Или, например, асинхронной можно сделать очередь и применять **await queue.get()**

Корутина (coroutine)

Корутинная функция – это функция, которая **может** приостанавливать выполнение (например, на **await**) и позже продолжать его с того же места.

await означает: “я сейчас жду, можно переключиться на другие задачи”

Корутинная функция:

```
async def handle_request():  
    await do_some_async_i/o()
```

Корутина (coroutine object):

```
coroutine = handle_request()
```

coroutine не выполнится сразу. Исполнение начнется, когда корутину запустит **event loop**, например, через **await**

Примечание:

await ожидает **awaitable** объект (есть **__await__**), не обязательно сразу **I/O**. Корутины могут вызывать друг друга по цепочке, но практический эффект есть, когда в итоге возникает **неблокирующее I/O** (HTTP, сокет, БД, таймер, очередь)

Почему I/O стал неблокирующим?

- **event loop** – управляет задачами (корутинами): планирует кого запустить, кого поставить на паузу, кого разбудить, делегирует задачи **selector'y** – планировщик-диспетчер задач
- Корутина доходит до **await socket.recv()** – **неблокирующий сокет**
 - а) Если данные в сокете уже есть – **recv()** сразу возвращает байты, корутина продолжает выполнение
 - б) Если данных нет – **recv()** сразу возвращает ошибку **EAGAIN**
- Как только вернулась **EAGAIN**, **selector** обрабатывает ошибку
 - **selector** – это часть инфраструктуры loop, наблюдает fd за событиями **READ** и **WRITE**
 - **selector** взаимодействует через system calls с ядром ОС:
 - **epoll_ctl(ADD/MOD, fd, EPOLLIN)** – регистрирует fd в **epoll** и указывает, какие события нас интересуют
 - **epoll_wait(epoll_fd, timeout)** – вызывает ожидание (блокировку) потока, пока не появятся готовые события или пока не истечет timeout. Возвращает список список fd, которые стали готовы к I/O (чтение/запись), и сообщает их event loop'у

Модель event loop

run_until_await_or_done() – выполняет task до ближайшего **await** или до завершения и возвращает событие ожидания

selector.register(...) – делает системный вызов **epoll_ctl(...)**

timeout = time_until_next_timer() – минимальное время до ближайшего таймера

selector.select(timeout) – делает системный вызов **epoll_wait(epoll_fd, timeout)**

```
while True:
    # 1) выполняем готовые задачи
    while ready_queue:
        task = ready_queue.pop()

        result = task.run_until_await_or_done()

        if result == DONE:
            continue

    elif result == WAIT_READ(fd):
        # сначала пробуем неблокирующее чтение
        try:
            data = nonblocking_recv(fd)
            task.set_recv_result(data)
            ready_queue.push(task)
        except EAGAIN:
            # данных пока нет -> регистрируем ожидание READ
            selector.register(fd, READ, data=task)

    elif result == WAIT_WRITE(fd):
        # сначала пробуем неблокирующую запись
        try:
            n = nonblocking_send(fd, task.pending_data)
            task.set_send_result(n)
            ready_queue.push(task)
        except EAGAIN:
            # сейчас писать нельзя -> ждём WRITE
            selector.register(fd, WRITE, data=task)

    # 2) если готовых задач нет - ждём I/O/таймер
    timeout = time_until_next_timer()
    events = selector.select(timeout)

    # 3) будим задачи, у которых fd стал ready
    for key, mask in events:
        task = key.data
        selector.unregister(key.fd)
        ready_queue.push(task)
```

asyncio

asyncio – это Python-фреймворк для написания асинхронных программ с использованием **async/await**

Предоставляет:

1. event loop
2. tasks (Task), futures (Future)
3. Примитивы координации: очередь, таймер

Позволяет эффективно работать с:

- сетевым I/O
- HTTP-запросами
- БД
- сокетами

Основные объекты asyncio

Task – это объект, который оборачивает корутину, управляет её выполнением и хранит результат – **контейнер для корутин**:

Свойства:

- хранит ссылку на корутину
- знает её состояние: pending, running, done, cancelled
- хранит результат или исключение
- может быть `await`-нута другой корутиной (есть `__await__`)

Future — это объект, представляющий результат, который появится позже – **контейнер для будущего результата**

Свойства

- хранит состояние: `pending` / `done`
- хранит результат или исключение
- может быть `await`-нут (есть `__await__`)
- сам ничего не выполняет
- обычно завершается извне через `set_result()` / `set_exception()`

```
import asyncio

task = asyncio.create_task(handle_request())
future = asyncio.create_future()
```

Немного погружения

- **`asyncio.run(demo())`** – создает event loop и запускает корутину `demo()` внутри одной Task
- **`loop.call_later(0.1, wsock.send, b"hello")`** – ставится таймер: через 0.1 сек отправить `b"hello"`
- **`loop.sock_recv()`**:
 - регистрирует сокет в selector
 - возвращает Future
 - Task приостанавливается
- Event loop:
 - ждет события (таймер + I/O)
 - таймер вызывает `wsock.send(b"hello")`
 - сокет становится readable
- Selector сообщает loop событие (event) → Future завершается → Task просыпается

В нутбуке мы сами двигали корутину, но за нас это делает event loop с помощью Task_step ->

```
async def child(rsock):
    loop = asyncio.get_running_loop()
    data = await loop.sock_recv(rsock, 1024)
    return data

async def main(rsock):
    x = await child(rsock)
    return x + b" + processed"

async def demo():
    loop = asyncio.get_running_loop()

    rsock, wsock = socket.socketpair()
    rsock.setblocking(False)
    wsock.setblocking(False)

    loop.call_later(0.1, wsock.send, b"hello")

    try:
        result = await main(rsock)
        print(result)
    finally:
        rsock.close()
        wsock.close()

asyncio.run(demo())
```

await task vs await coroutine

await cor – это inline-await корутины в текущей task

create_task(cor) – это превращение корутины в отдельную task, которую loop может планировать конкурентно с текущей

await task – это просто ожидание результата этой уже запущенной отдельной task.

Типичный пример с async

Пусть есть приложение с сетевыми запросами от нескольких юзеров одновременно.
И пусть три юзера: А, В, С

```
async def handle_request(user_id):  
    data = await fetch_from_api(user_id) # HTTP / БД / любая  
    неблокирующая операция  
    ...  
    return data
```

Тогда для каждого юзера создается своя **корутина**, которая оборачивается в свою **Task**:

- Юзер А: **await fetch_from_api** – в этот момент выполнение приостанавливается до завершения I/O-операции (ждем сеть)
- Пока I/O юзера А в ожидании, event loop продолжает выполнять другие задачи, которые готовы к выполнению, например, начинает обрабатывать запросы от других юзеров
- Когда результат приходит, соответствующая корутина снова становится готовой и продолжает выполнение с точки после **await**

run in executor

В конце вложенных вызовов корутин может быть вызов sync-задачи, которая может заблокировать текущий поток.

Тогда ее можно запустить через:

```
event_loop.run_in_executor(executor, func, *args),
```

где executor – **Thread/Process PoolExecutor**, func – блокирующая операция

В итоге event loop не заблокируется, т.к. синхронная функцию будет выполняться в другом потоке (пуле потоков)

Гонка состояний

За счет того, что `asyncio` переключает задачи (корутины) только в точках `await`, поэтому доступ к общему ресурсу легче контролировать, по сравнению с гонкой в многопоточности

Тем не менее, в `asyncio` есть свои примитивы синхронизации

- `asyncio.Lock`
- `asyncio.Semaphore`

Когда они могут понадобиться

- несколько корутин работают с **общим изменяемым объектом** (`dict`, `list`, счётчик, кэш)
- несколько корутин используют **общий ресурс**, например, одно соединение (`connection`) с БД, один сокет, один файл

async vs threading vs processing

async – всегда concurrency

threading – concurrency, но может и чистый parallelism

processing – parallelism

Quiz

Coroutine await